

Lightning Web Components

Modern, standards-based web components for building fast Salesforce UI.

• Component Anatomy

Three files share the folder + base name:

```
myCmp/  
  myCmp.js           // controller  
  myCmp.html         // template  
  myCmp.js-meta.xml // config & targets
```

- `.css` and `.svg` are optional.
- Reference in markup as `<c-my-cmp>` (kebab-case).

• JS Module

```
import { LightningElement, api, wire }  
  from 'lwc';  
  
export default class MyCmp  
  extends LightningElement {  
  @api recordId;    // public, set by parent  
  greeting = 'Hi';  // reactive by default  
}
```

• Decorators

- `@api` – public, reactive property/method.
- `@wire` – bind to a data adapter or Apex.
- `@track` – deep-track object/array fields (rarely needed now – fields are reactive by default).

• Template Directives

```
<template lwc:if={ready}>  
  <p>{greeting}</p>  
</template>  
<template for:each={items} for:item="row">  
  <li key={row.id}>{row.name}</li>  
</template>
```

- `lwc:if` | `lwc:elseif` | `lwc:else` (modern).
- `for:each` + a unique `key`; `iterator`: adds `first` / `last`.
- `lwc:ref="box"` → `this.refs.box` for DOM access.

• Wire a Record

```
import { getRecord } from  
  'lightning/uiRecordApi';  
import NAME from  
  '@salesforce/schema/Account.Name';  
  
@wire(getRecord,  
  { recordId: '$recordId', fields: [NAME] })  
account;
```

- `'$prop'` makes the wire **reactive** to that property.
- Result is an object – read `account.data` / `account.error`.

• Call Apex

```
import getList from  
  '@salesforce/apex/AcctCtrl.getList';  
  
// Reactive  
@wire(getList, { ind: '$industry' }) accts;  
  
// Imperative  
getList({ ind: 'Tech' })  
  .then(data => this.rows = data)  
  .catch(err => this.error = err);
```

- Method needs `@AuraEnabled(cacheable=true)` to wire.

• Lifecycle Hooks

HOOK	FIRES
<code>constructor()</code>	instance created
<code>connectedCallback()</code>	inserted in DOM
<code>renderedCallback()</code>	after every render
<code>disconnectedCallback()</code>	removed
<code>errorCallback(e, s)</code>	descendant error

• Events

```
this.dispatchEvent(  
  new CustomEvent('select', {  
    detail: { id }, bubbles: true  
  })  
);  
  
<c-child onselect={handleSelect}></c-child>
```

- Across the DOM: **Lightning Message Service**.

• Navigation

```
import { NavigationMixin }  
  from 'lightning/navigation';  
  
this[NavigationMixin.Navigate]({  
  type: 'standard__recordPage',  
  attributes: {  
    recordId, objectApiName: 'Account',  
    actionName: 'view'  
  }  
});
```

• Lightning Data Service

```
<lightning-record-form  
  record-id={recordId}  
  object-api-name="Account"  
  fields={fields}>  
</lightning-record-form>
```

- No Apex; respects FLS & sharing automatically.
- Also `lightning-record-edit-form`, `getRecord`.

• Config (meta.xml)

```
<isExposed>true</isExposed>  
<targets>  
  <target>lightning__RecordPage</target>  
  <target>lightning__AppPage</target>  
</targets>
```

• Base Components

- `lightning-button` · `lightning-input`
- `lightning-card` · `lightning-datatable`
- `lightning-spinner` · `lightning-icon`
- `lightning-combobox` · `lightning-tabset`